

DataBase And Management
Systems

ASSIGNMENTS

Name : Dev M.K

Reg No: 61782324111014

Dept/Sec: CSE(AIML) - "A"

ASSIGNMENT - I

(1)

Consider the employee database, what are the appropriate Primary Keys?

employee (person-name, Street, City)

works (person-name, Company-name, Salary)

Company (Company-name, City)

Tables (Schemas).

1. employee.

Employee (person-name, Street, City)

2. works.

works (Person-name, Company-name, Salary)

3. Company

Company (company-name, city)

Appropriate Primary Keys.

1. Employee relation.

Primary Key: Person-name.

Reason: Each employee is uniquely identified by their name in the data base.

2. Company relation.

Primary Key: Company-name

Reason: Each Company has a unique name.

3. works relation.

Primary Key: (Person-name, Company-name)

Reason: A Person works for more than one Company.
A Company may have many employees.

(2)

The combination of Person-name and Company-name uniquely identifies each tuple.

2. Consider the Foreign-Key Constraint from the dept-name attribute of instructor to the department relation. Give examples of inserts and deletes to these relations that can cause a violation of the Foreign-Key Constraint.

Give Relations -

1. department.

department (dept-name, building, budget)

2. instructor.

instructor (ID, name, dept-name, salary)

Foreign-Key Constraint.

instructor, dept-name references.

department - dept-name.

a) insert that causes a foreign key violation.

department.

dept-name	building	budget.
-----------	----------	---------

Comp. Sci	Watson	100000
-----------	--------	--------

physics	Newton	90000
---------	--------	-------

Invalid insert into instructor.

> Sql

insert into instructor.

values (101, 'Ram', 'Astro', 70000);

Why this is a violation:

The department 'Astro' does not exist in the department relation.

Hence, the foreign-key constraint is violated

b) delete that causes a foreign key violation existing instructor table.

Instructor.

ID	name	dept-name	Salary
----	------	-----------	--------

22222	Einstein	physics	95000
-------	----------	---------	-------

delete from department

> Sql

delete from department.

where dept-name = 'physics';

Why this is a violation.

The department 'physics' is still referenced by the instructor table

- Deleting it would have a dangling reference in instructor

- Hence, the foreign-key constraint is violated.

3. Consider the time slot relation. Given that a particular time slot can meet more than once in a week, explain why day and start time are part of the primary key of this relation, while end time is not.

Time slot Relation.

time slot id	Day	Start time	end time.
A	M	8:00	8:50
A	W	8:00	8:50
A	F	8:00	8:50
B	M	9:00	9:50
B	W	9:00	9:50
B	F	9:00	9:50
C	M	11:00	11:50
C	W	11:00	11:50
C	F	11:00	11:50
D	M	13:00	13:50
D	W	13:00	13:50
D	F	13:00	13:50
E	T	10:30	11:45
E	T	14:30	11:45
E	R	16:00	15:45
F	T	16:00	15:45
F	R	10:00	16:50
G	M	16:00	16:50
G	W	16:00	16:50
G	F	16:00	16:50
H	W	10:00	12:30

Primary Key of time-slot.

(5)

Primary Key.

(time-slot-id, day, start-time)

Explanation:

A single time-slot id can occur multiple times in a week.

Eg: Time slot a occur on monday, wednesday and Friday at 8:00

Therefore time-slot-id alone cannot uniquely identify a tuple.

The day attribute is required to distinguish different occurrences of the same time slot on different days.

The start-time is required because two slots on the same day may start at different times.

Why end-time is not part of the primary key.
end-time is functionally dependent on start-time and time-slot-id.

Once the start-time is known, the end-time is automatically determined.

Including end-time in the primary key would be redundant.

A. In the instance of instructor shown in figure 2.1 (a) two instructors have same name from this, can we conclude that name can be used as Superkey (or) primary key of instructor.

ID	name	dept - name	Salary
10101	Srinivasan	Comp. Sci	65000
12121	Wu.	Finance	90000.
15151	Mozare	Music	40000.
22222	Einstein	physics	95000.
32343	EI Said	History	60000.
33456	Gold	Physice.	87000.
45565	Katz	Comp. Sci	75000.
58583	Califien	History	62000
76543	Singh	Finance	80000.
76766	Crice	Biology	72000.
83821	Brandt	Comp Sci	92000
98845	Kim	Elec. Eng.	80000.

Answer :

Short answer : no, we cannot conclude that name can be used as a Superkey or primary key.

Explanation :

In the given instance of the instructor relation, it happens.

that no two instructors have the same name.

However, a Superkey or primary Key must uniquely identify tuples in all possible valid instances of the relation not just in one snapshot of data.

Why name is not suitable as a Key:

1. names are not guaranteed to be unique in real-world scenarios, two instructors can have the same level. (e.g... two instructors named Singh or bin)
2. Keys are based on semantics, not current data just because name is unique in the current table does not mean it will remain unique after future inserts
3. Primary keys must be stable and minimal names can change (Spelling corrections, name changes)
ID is designed to uniquely and permanently identify an instructor.

5. What is the result of first performing the Cartesian Product of Student and advisor, and then performing a Selection operation on the result with the predicate $S_{id} = 10$? (Using the symbolic notation of relational algebra, this query can be as

$$\sigma_{S_{id} = 10} (Student \times advisor)$$

ID	name	dept. name	tot. cred.
00128	Xhang	Comp Sci	102
12345	Shankar	Comp. Sci	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Palties	Physics	56
45678	Uuy	physics	46
54321	Williams	Comp Sci	54
55739	Sanchenx	music	38
70557	Shou	physics	0
76543	Prooun	Comp. Sci	58
76653	Aoi	Elec. Eng.	60
98765	Bouniteas	Elec. Eng.	98
98988	Tanaka	biology	120

S. id	i. id
00128	45565
12345	10101
23121	76543
44553	22222
45678	45565
16543	98345
76653	98345
98765	76766
98988	76766.

Steps: Cartesian Product (Student $\times$ advisor) (9)

This combines every tuple of Student with every tuple of advisor.

If there are m Students and n advisor tuples the result has $m \times n$ tuples.

Attribute becomes:

Student: ID, name, dept-name, tot-cred,
advisor: s-id, advisor-i-id.

Step 2: Selection σ ($S-id = ID$)

Student-ID: advisor-s-id.

This matches each student with their advisor.

Effectively, this behaves like an equi-join b/w Student & advisor.

Final result:

ID (Student)	name	dept-name	tot-cred.	S-id	P-id
00128	Zhang	Comp. Sci	102	00128	45565
12345	Shankar	Comp. Sci	32	12345	10101
23121	Chavez	Finance	110	23121	16543
44553	Pelfier	Physics	56	44553	22222
45678	Lery	Physics	46	45678	22222
76543	Brown	Comp. Sci	58	76543	45565
76653	Adi	Elec. Eng.	60	98765	98345
98765	Bouniees	Elec. Eng.	98	98765	98345
98988	Tanaka	Biology	120	98988	976766

Consider the employee database of figure 2.17 give ⁽¹⁰⁾ an expression in the relational algebra to express each of the following queries:

a) find the name of each employee who lives in city "miami".

b) find the name of each employee whose salary is $> \$100000$.

c) find the name of each employee who lives in "miami" and whose salary $> \$100000$.

a. Π employee-name (σ city = "miami" (employee))

Explanation:

$\Rightarrow$ Select employees whose city is miami

$\Rightarrow$ Project only their names.

b. Π employee-name (σ salary > 100000 (works))

Explanation:

$\Rightarrow$ Select tuples from works with salary greater than 100000.

$\Rightarrow$ project employees names.

c. Π employee-name (σ city = "miami" $\wedge$ salary > 100000 (employee $\bowtie$ works))

Explanation:

Join employee and works on employee-name
Select employee who live in miami and earn more than 100000.

Project only the employee names.

a) Find the name of each branch located in Chicago. (11)

Π branch-name (σ branch-city = 'Chicago' (branch))

b) Find the ID of each borrower who has a loan in branch "down town".

Π ID (borrower $\bowtie$ borrower, loan, number = loan, loan, number σ branch-name = 'Down town' (loan))

Consider the employee database given an expression in the relational algebra to express each of the following queries.

a) Find the ID and name of each employee who does not work for "Big Bank".

b) Find the ID and name of each employee who earns at least as much as every employee in the database.

a) Select employee-id, employee-name
From employee

Where employee-id NOT IN

Select employee-id

From works

Where Company-name = "Big Bank"

);

b) Select e-employee-id, e-employee-name
from employee e
join works w on e-employee-id = w.employee-id.
Where w. Salary >= ALL (

Select Salary
from works.

);

9) The division operator of relational algebra, " $\div$ ", is defined as follows. let $r(R)$ and $s(S)$ be relations and let $S \subseteq R$; that is, every attribute of schema S is also in Schema R . Given a tuple t , let $t(S)$ denote the projection of tuple t on the attribute in S . Then $r \div s$ is a relation on Schema $R-S$ (that is, on the Schema containing all attribute of Schema R that are not in Schema S). A tuple t is a in $r \div s$ if and only if both of two conditions hold:

- t is in $UR - S(x)$
- For every tuple ts in s , there is a tuple tr in r satisfying both of the following:

a. $tr[S] = ts[S]$

b. $tr[R-S] = t$

that .

division operator ($\div$) in Relational Algebra.

* $r(R)$ and $s(S)$ are relations

* $S \subseteq R$ (all attributes of S are also present in R)

Result Schema.

13

$$r \div s$$

* The result relation has attributes.

$$R - S$$

* all attributes of R except those in S
meaning of $r \div s$.

A tuple t belongs to $r \div s$ if and only if the following two conditions are satisfied.

Condition 1

$$t \in \pi_{R-S}(r)$$

Condition 2

For every tuple $t_s \in s$, there must exist a tuple $t_r \in r$ such that

a) $t_r[s] = t_s[s]$

b) $t_r[R-S] = t$

eg:

Student-id	Course-id	$S(\text{course-id})$
1	CS101	CS101
1	CS102	CS102
2	CS101	
2	CS101	
3	CS101	

General Relational Algebra expression for division ⁽¹⁴⁾

Relation $r(R)$

Relation $S(S)$ where $S \subseteq R$.

division query:

$$r \div S = \pi_{R-S}(r) - \pi_{R-S}((\pi_{R-S}(r) \times S) - r)$$

10) a) Write a relational algebra expression using the division operator to find the IDs of all students who have taken all Comp. Sci. Courses and generate the set of all Comp. Sci. Courses - ids. using a Select expression, before doing the division

b) Show how to write the above query in relational algebra, without using division. So you would have shown how to define the division operation using the other relational algebra operations.

Give relations (Std University Schema)

- takes (ID, course-id, sec-id, sem, year, grade)
- Course (course-id, title, dept-name, credits)

a) Using the division ($\div$) operator.

Goal:

Find the IDs of all student who have taken all Comp. Sci. Courses.

Step 1: Get only (ID, course-id) from takes

$$r = \pi_{ID, \text{course-id}}(\text{takes})$$

Step 2: Get all Comp. Sci. course - ids.

(15)

$$S = \pi_{\text{course-id}} (\sigma_{\text{dept-name} = \text{"Comp. Sci."}} (\text{course}))$$

Step 3: Apply Division.

$$r \div s$$

Final answer:

$$\pi_{\text{id}, \text{course-id}} (\text{takes}) \div \pi_{\text{course-id}} (\sigma_{\text{dept-name} = \text{"Comp Sci"}} (\text{course}))$$

b) without using the division operator.

division definition used:

$$r \div s = \pi_{R-S} (r) - \pi_{R-S} ((\pi_{R-S}(r) \times s) - r)$$

Apply it to this problem.

$$R = \pi_{\text{id}, \text{course-id}}$$

$$S = \text{course-id}$$

$$R-S = \pi_{\text{id}}$$

$$\pi_{\text{id}} (\text{takes}) - \pi_{\text{id}} ((\pi_{\text{id}} (\text{takes}) \times \pi_{\text{course-id}} (\sigma_{\text{dept-name} = \text{"Comp. Sci."}} (\text{course}))) - \text{takes})$$

$$\pi_{\text{id}, \text{course-id}} (\text{takes})$$

Consider the advisor relation shown in the schema diagram with sid as the primary key of advisor. Suppose a student can have more than one advisor, then would sid still be a pri-key of the advisor relation?

If not, what should the primary key of advisor be?

takes

ID
course-id
sec-id
Semester,
year, grade

Student

ID
 name
 dept-name
 tot. cred.

Section
 course-id
 sec-id
 semester
 year
 building
 room-number
 time-slot

time-slot
 time-slot-id
 day
 start-time
 end-time

Course
 course-id
 title
 dept-name
 credits
 department
 dept-name
 building
 budget

advisor
 s-id
 i-id

class room
 building
 room-number
 capacity

teaches

ID
 course-id
 sec-id
 semester
 year

prereq
 course-id
 prereqid

instructor
 ID
 name
 dept-name
 salary

no.

In the given schema, the advisor relation is
 advisor (s-id, i-id)

Originally, $S-id$ is the primary key,

(17)

each student can have at most one advisor

When a student can have more than one advisor

The same $S-id$ can appear multiple times in the advisor (with different $I-id$ values)

Therefore, $S-id$ alone is no longer unique

Hence, $S-id$ can't be the primary key

($S-id$, $I-id$)

($S-id$, $I-id$)

Final Answer:

No, $S-id$ would not be a primary key if a student can have more than one advisor

The primary key of the advisor relation should be the composite key ($S-id$, $I-id$)

Reason:

The pair ($S-id$, $I-id$) uniquely identifies each advisor-student relationship.

Consider the bank database of Figure. Assume that branch names and customer names uniquely identify branches and customers, but loans and accounts can be associated with more than one customer. What are the appropriate primary keys?

b) Given your choice of primary keys, identify appropriate foreign keys. (10)

Bank database relations -

Branch (branch-name, branch-city, assets)

Customer (Customer-name, Customer-street, Customer-city)

Loan (loan-number, Customer-street, Customer-city, branch-name, amt);

account (account-number, branch-name, balance)

borrower (Customer-name, loan-number)

depositor (Customer-name, account-number)

a) Approx primary keys.

Branch names uniquely identify branches

Customer names uniquely identify Customers.

loan and account can be associated with more than one customer.

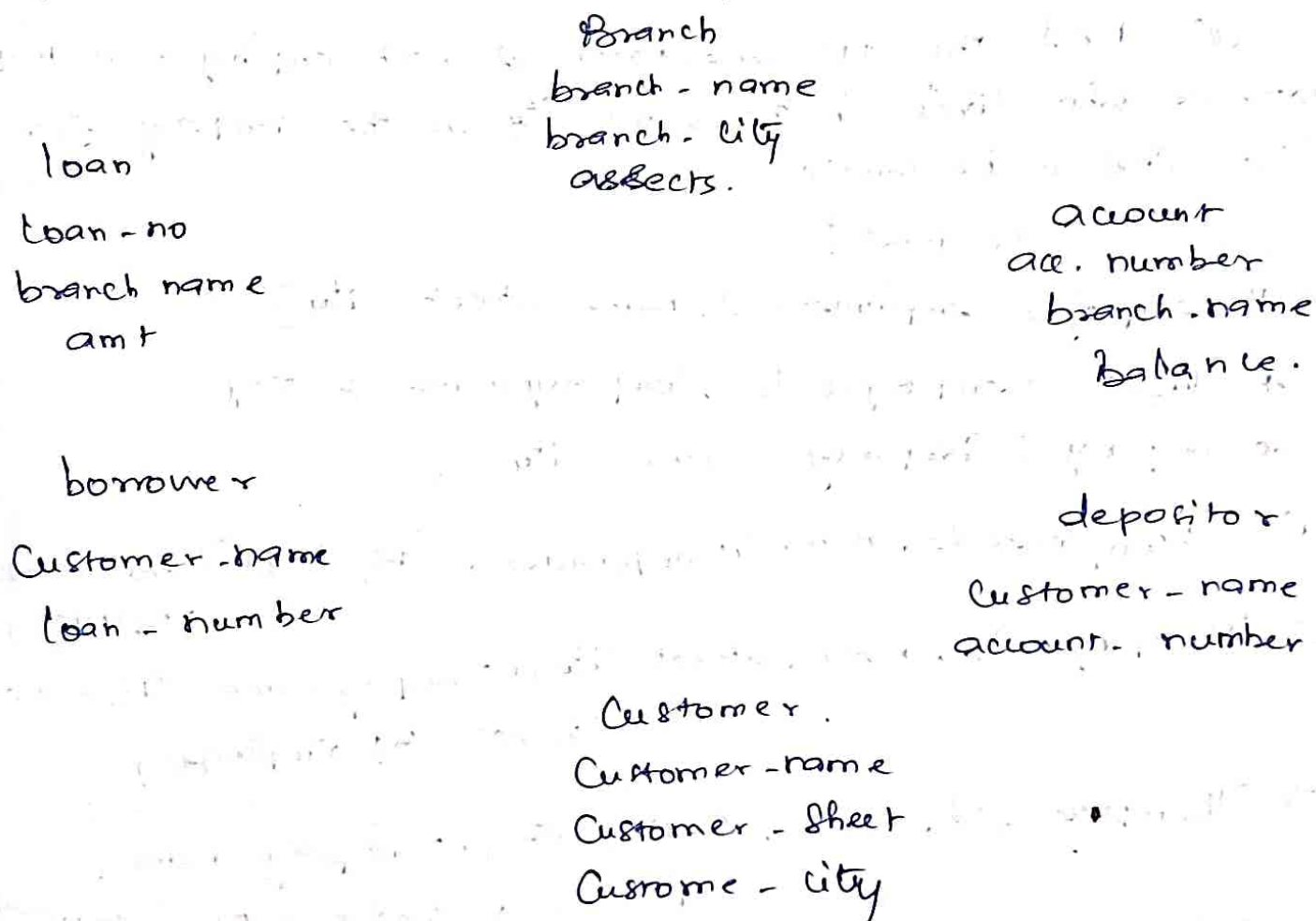
Primary keys:

Relation	PK-key
branch	branch-name
Customer	Customer-name
loan	loan.no.
account	acc.no.
borrower	(Customer-name, loan-number)
depositor	(Customer-name, account-number)

b) Approx Foreign Keys :

Relation	Foreign Key	References
loan	branch . name	branch (branch . name)
account	branch . name	branch (branch . name)
borrower	Customer . name	Customer (Customer . name)
borrower	loan - no .	loan (loan . number)
depositor	Customer . name	customer (Customer . name)
depositor	account - number	account (account . number)

13) Construct a schema diagram for the bank database



Consider the employee database Give an expression in the relational algebra to express each of the following queries:

a) Find the ID and name of each employee who works for "Big Bank".

b) Find the ID, name, and city of residence of each employee who works for "Big Bank".

c) Find the ID, name, street address and city of residence of each employee who works for "Big Bank" and earns more than \$10000.

d) Find the ID and name of each employee in this database who lives in the same city as the company for which she or he works.

Schemes assumed:

* employee (employee-id, name, street, city)

* works (employee-id, company-name, salary)

* Company (company-name, city)

a) $\pi_{\text{employee-id, name}}(\sigma_{\text{company-name} = \text{"Big Bank"}}(\text{works}) \bowtie \text{employee})$

b) $\pi_{\text{employee-id, name, street, city}}(\sigma_{\text{company-name} = \text{"Big Bank"}}(\text{works}) \bowtie \text{employee})$

c) $\pi_{\text{employee-id, name, street, city}}(\sigma_{\text{company-name} = \text{"Big Bank"} \wedge \text{salary} > 10000}(\text{works}) \bowtie \text{employee})$

d) $\pi_{\text{employee-id, name}}(\sigma_{\text{employee-city} = \text{company-city}}(\text{employee} \bowtie \text{works} \bowtie \text{company}))$

Consider the bank database of figure. Give an expression in the relational algebra for each of the following queries.

- Find each loan number with a loan amt greater than \$ 10000.
- Find the ID of each depositor who has an account with a balance greater than \$ 6000.
- Find the ID of each depositor who has an account without balance $>$ \$ 6000 at the "uptown" Branch.

Schemes assumed.

- * Loan (loan-number, branch-name, amount)
- * account (account-number, branch-name, balance)
- * borrower (customer-id, loan-number)
- * depositor (customer-id, account-number)

- $\pi_{\text{loan-number}} (\sigma_{\text{amount} > 10000} (\text{loan}))$
- $\pi_{\text{customer-id}} (\sigma_{\text{balance} > 6000} (\text{account}) \bowtie \text{depositor})$
- $\pi_{\text{customer-id}} (\sigma_{\text{balance} > 6000 \wedge \text{branch-name} = \text{"Uptown"}} (\text{account}) \bowtie \text{depositor})$

List two reasons why null values might be introduced (22)
be introduced into a database.

Two common reasons why null values are introduced into a database are:

1. Missing or unknown information.

* The values exist but is not known at the time of data entry.

eg: A customer's phone number is not yet available.

2. Inapplicable attributes:

* The attribute does not apply to a particular tuple.

eg: spouse name for an unmarried employee

Discuss the relative merits of imperative, functional and declarative languages.

1. IMPERATIVE : LANGUAGES.

eg: C, java, python, pascal.

merits:

* intuitive and natural: closely match how computers execute instructions step by step.

* Fine-grained control: programmer controls memory, variables, and execution flow.

* efficient execution: often allows optimized performance

* widely used: large ecosystem, tools, libraries, available.

Demerits :

* Programs can become complex and error prone due to mutable state.

* Harder to reason about correctness, especially in large program

* not well studied for expressing high-level logic concisely

FUNCTIONAL LANGUAGES :

eg. (Haskell, Lisp, Scheme, ML)

Merits :

* Mathematical clarity : Based on functions and expressions.

* no side effects (or minimal) : easier to test, debug and reason about.

* Supports parallelism well : Immutability avoids data race issues

* Program. are often shorter and more elegant.

Demerits :

* Steeper learning curve, especially for programmers use to imperative style.

* Less intuitive for tasks involving state changes.

* Sometimes less efficient for low-level system programming.

DECLARATIVE LANGUAGES :

(eg. SQL, Prolog, HTML)

Merits :

* Focus on what needs to be done, not how to do it.

* Very Concise and readable.

* System handles optimization and execution

Strategy.

* Ideal for database querying and knowledge representation

Demerits :

* Limited control over execution details.

* not suitable for general-purpose programming.

* Performance depends heavily on the underlying implementation.

Write the following queries in relational algebra, using the University Scheme.

a) Find the ID and name of each instructor in the physics department.

b) Find the ID and name of each instructor in a department located in the building "Watson".

c) Find the ID and name of each student who has taken at least one course in the 'Comp. Sci' department.

d) Find the ID and name of each student who has taken at least one course section in the year 2018.

e) Find the ID and name of each student who has not taken any course section in the year 2018.

Schemes assumed.

* instructor (ID, name, dept-name, salary)

* department (dept-name, building, budget)

* student (ID, name, dept-name, tot-cred)

* course (course-id, title, dept-name, credits)

* section (course-id, sec-id, semester, year, building, room-number, time-slot-id)

* takes (ID, course-id, sec-id, semester, year, grade)

a) $\Pi_{ID, name} (\sigma_{dept-name = 'physics'} (instructor))$

b) $\Pi_{ID, name} (instructor \bowtie_{building = 'Watson'} (department))$

c) $\Pi_{ID, name} (student \text{ } \not\bowtie \text{ } \text{takes } \bowtie_{dept-name = 'Comp. Sci'} (course))$

d) $\Pi_{ID, name} (Student \bowtie_{\text{year} = 2018} (takes))$

e) $\Pi_{ID, name} (Student) - \Pi_{ID, name} (Student \bowtie_{\text{year} = 2018} (takes))$